

Diseño de Bases de Datos: Arquitectura Híbrida para Ecommify

Actividad 2 — PostgreSQL Avanzado

Tipos Nativos · Extensiones · Modelado OLTP/OLAP

Grupo:

Andres Camilo Meneses Ortega
David Hernando Monsalve Delima
Eduardo Trujillo Santos
Nicolás Cifuentes Barriga

Maestría en Arquitectura de Software, Universidad de la Sabana

Diseño y Optimización de Bases de Datos MAS 2026-3 G1G2

Miguel Alfonso Varela

2026

1. Exploración de Tipos de Datos Nativos en PostgreSQL

La Actividad 1 estableció el esquema normalizado de Ecommify en 3FN con nueve tablas relacionales. Esta sección extiende ese esquema incorporando tipos de datos avanzados de PostgreSQL que permiten modelar información semiestructurada, colecciones, rangos temporales y estructuras compuestas sin abandonar el modelo relacional. Cada tipo se justifica contra una alternativa más simple, considerando el patrón de acceso real del dataset Olist (99,441 pedidos, 32,951 productos, 71 categorías).

1.1 Inventario de Tipos Avanzados y Casos de Uso

PostgreSQL ofrece un conjunto de tipos nativos que van más allá de los tipos escalares básicos. La siguiente tabla documenta los tipos evaluados para Ecommify, su definición y su aplicación concreta en el dominio:

Tipo Avanzado	Definición	Indexación recomendada
JSONB	Binario JSON indexable	GIN index sobre claves específicas
ARRAY[]	Colección ordenada de un tipo	GIN index + operadores @>, &&
TSTZRANGE	Rango temporal con timezone	GIST index, operadores &&, @>
NUMRANGE	Rango numérico continuo	GIST index, contained by
hstore	Clave-valor plano	GIN/GIST index, operadores ->
Composite Type	Tipo estructurado reutilizable	Acceso por campo: (address).street

Tabla 1. Tipos de datos avanzados de PostgreSQL aplicados al dominio Ecommify

1.2 Comparación JSONB vs. Columnas Normalizadas

La decisión más frecuente en modelado híbrido es cuándo usar JSONB en lugar de columnas relacionales. Esta comparación se aplica al caso concreto de los atributos de producto en Ecommify, donde el dataset Olist incluye 71 categorías con atributos físicos heterogéneos:

Criterio	JSONB	Columnas Normalizadas
Esquema flexible	JSONB ✓ — Cada categoría tiene atributos distintos (electrónico vs. ropa)	Columnas ✗ — Requiere ALTER TABLE por cada nuevo atributo; columnas NULL abundan
Rendimiento lectura punto	JSONB ✓ — Index GIN sobre product_id es O(log n)	Columnas ✓ — Acceso directo por columna indexada
Consultas analíticas	JSONB ~ — jsonb_to_recordset() agrega pero es más costoso	Columnas ✓ — JOIN y GROUP BY eficientes con índices B-tree
Evolución del esquema	JSONB ✓ — Nuevo atributo: solo insertar en el documento	Columnas ✗ — DDL en producción requiere lock de tabla

Tabla 2. Comparación JSONB vs. columnas normalizadas para atributos de producto

1.3 Transformación desde el Dataset Olist

Cada bloque muestra las columnas base del dataset que se agrupan en el nuevo tipo y el código equivalente.

```
-- JSONB: agrupación de dimensiones físicas del producto
ALTER TABLE products
  ADD COLUMN IF NOT EXISTS product_specifications JSONB DEFAULT '{} '::jsonb;

-- Poblar desde columnas existentes del dataset
UPDATE products SET product_specifications = jsonb_build_object(
  'weight_g', product_weight_g,
  'length_cm', product_length_cm,
  'height_cm', product_height_cm,
  'width_cm', product_width_cm
);

-- TSTZRANGE: ciclo de vida del pedido (compra → entrega estimada)
ALTER TABLE orders
  ADD COLUMN IF NOT EXISTS order_lifecycle TSTZRANGE;

UPDATE orders SET order_lifecycle = tstzrange(
  order_purchase_timestamp,
  order_estimated_delivery_date,
  '[]'
);
```

```

-- TSVECTOR: índice de búsqueda textual sobre reseñas
ALTER TABLE order_reviews
  ADD COLUMN IF NOT EXISTS review_search TSVECTOR
  GENERATED ALWAYS AS (
    to_tsvector('portuguese',
      COALESCE(review_comment_title, '') || ' ' ||
      COALESCE(review_comment_message, '')
    )
  ) STORED;

CREATE INDEX idx_reviews_search_gin
  ON order_reviews USING GIN (review_search);

-- POINT (PostGIS GEOMETRY)

ALTER TABLE geolocation
  ADD COLUMN IF NOT EXISTS geo_point GEOMETRY(Point, 4326);

UPDATE geolocation SET geo_point = ST_SetSRID(
  ST_MakePoint(geolocation_lng, geolocation_lat), 4326
);

-- HSTORE: metadata de pago como pares clave-valor
ALTER TABLE order_payments
  ADD COLUMN IF NOT EXISTS payment_metadata HSTORE;

UPDATE order_payments SET payment_metadata = hstore(ARRAY[
  'type',          payment_type,
  'installments', payment_installments::TEXT,
  'value',         payment_value::TEXT
]);

```

1.4 Tabla de Decisión: Qué Campo, Qué Tipo y Por Qué

La siguiente tabla sintetiza las decisiones de diseño con su justificación técnica, incluyendo tanto los tipos definidos en la arquitectura (Sección 1.3) como los derivados directamente de las columnas del dataset Olist en Colab:

Tabla	Campo	Tipo Avanzado	Justificación técnica
Tipos definidos en la arquitectura — Sección 1.3			
products	product_specifications	JSONB	Agrupar peso y dimensiones físicas (weight_g, length_cm, height_cm, width_cm) en un documento flexible. Permite añadir atributos de categoría (talla, material, voltaje) sin ALTER TABLE. GIN index habilita búsqueda por clave específica.
products	product_photos_urls	TEXT[]	Lista ordenada de URLs de imágenes. El primer elemento es la foto principal. Operador @> filtra productos con al menos N fotos. Elimina tabla auxiliar product_photos y el JOIN extra asociado.
products	promotion_period	TSTZRANGE	Vigencia de descuento con timezone. promotion_period @> NOW() consulta promociones activas con GIST index. Reemplaza promo_start / promo_end y garantiza que end >= start por construcción del tipo.
order_items	price_range_tier	NUMRANGE	Segmento de precio en el momento de compra ([0,50), [50,200), [200,∞)). Habilita consultas analíticas de rango sin CASE. GIST index optimiza containment queries en reportes de ticket promedio.
sellers	seller_metadata	hstore	Metadatos operativos planos (tiempo_respuesta_hrs, idioma_soporte, certificaciones). Más eficiente que JSONB para pares clave-valor sin anidamiento. Un campo se actualiza sin reescribir el documento completo.
customers	delivery_address	Composite Type	Tipo address_t(street, number, complement, zip) reutilizado en sellers. Agrupa campos relacionados; acceso: (delivery_address).zip. Reduce proliferación de columnas y garantiza consistencia estructural.
Transformaciones desde columnas del dataset Olist — Sección 1.4 (Colab)			
orders	order_lifecycle	TSTZRANGE	Intervalo natural del pedido: desde order_purchase_timestamp hasta order_estimated_delivery_date. Permite consultas de solapamiento de períodos de entrega y cálculo de duración estimada con un solo campo.

Tabla	Campo	Tipo Avanzado	Justificación técnica
order_reviews	review_search	TSVECTOR	Concatena review_comment_title y review_comment_message en un vector de búsqueda full-text en portugués. Columna GENERATED STORED mantiene el índice GIN sincronizado automáticamente sin lógica de aplicación.
geolocation	geo_point	POINT / GEOMETRY	Representa latitud y longitud como punto geoespacial nativo (PostGIS). Habilita ST_Distance y ST_DWithin sobre 1,000,163 registros. Elimina el cálculo de distancia en capa de aplicación y reduce tráfico de datos.
order_payments	payment_metadata	hstore	Agrupar payment_type, payment_installments y payment_value como pares clave-valor. Útil para metadatos de pago extendidos sin columnas adicionales. Consistente con seller_metadata: ambos usan hstore para datos planos de bajo volumen de escritura.

Tabla 3. Justificación de tipo avanzado por campo — Ecommify / Dataset Olist

2. Análisis de Extensiones de PostgreSQL

Las extensiones de PostgreSQL permiten incorporar tipos, funciones e índices especializados sin cambiar el motor de base de datos. Para Ecommify, la Actividad 1 identificó el uso de PostGIS para geolocalización y la arquitectura híbrida con MongoDB para datos semiestructurados. Esta sección evalúa cinco extensiones bajo tres criterios: **relevancia para el dominio**, **impacto en rendimiento** y **costo de implementación y mantenimiento**.

2.1 Inventario de Extensiones Evaluadas

La evaluación parte de las necesidades identificadas en la Actividad 1: datos geoespaciales en la tabla geolocation, búsqueda textual en el catálogo de productos.

Extensión	Funcionalidad	Caso de uso Ecommify	Relevancia
PostGIS	Tipos y funciones geoespaciales (puntos, polígonos, distancias)	Cálculo de distancia vendedor-cliente para optimizar costos de flete y rutas de entrega	Alta — geolocation tiene lat/lng para 1,000,163 registros
pg_trgm	Trigramas para similitud de texto y búsqueda tolerante a errores	Búsqueda de productos con tolerancia a errores tipográficos en nombre y categoría	Alta — catálogo con 32,951 productos en portugués
hstore	Almacenamiento clave-valor en columna simple	Metadatos operativos de vendedores sin esquema fijo (ya implementado en Sección 1)	Media — complementa JSONB para datos planos
pgcrypto	Funciones criptográficas (hash, cifrado simétrico/asimétrico)	Protección de customer_unique_id y datos sensibles de pago en reposo	Protección de datos
pg_stat_statements	Estadísticas de ejecución de queries para diagnóstico de rendimiento	Identificar queries lentos en OLTP antes de crear índices adicionales	Media — imprescindible en fase de optimización

Tabla 4. Extensiones PostgreSQL evaluadas para Ecommify

2.2 PostGIS: Optimización de Costos de Envío

El dataset Olist contiene coordenadas geográficas de 1,000,163 registros de geolocalización. La Actividad 1 identificó que sellers y customers referencian geolocation por zip_code_prefix. PostGIS permite calcular la distancia real entre vendedor y cliente, base para tarifas de flete dinámicas:

```

CREATE EXTENSION IF NOT EXISTS postgis;

-- Agregar columna de geometría a geolocation
ALTER TABLE geolocation
  ADD COLUMN geom GEOMETRY(Point, 4326);

-- Poblar geometría desde lat/lng existentes
UPDATE geolocation
  SET geom = ST_SetSRID(
    ST_MakePoint(geolocation_lng, geolocation_lat), 4326
  );

-- Índice GIST para búsquedas espaciales eficientes
CREATE INDEX idx_geolocation_geom
  ON geolocation USING GIST (geom);

-- Función: distancia en km entre vendedor y cliente de un pedido
CREATE OR REPLACE FUNCTION fn_shipping_distance_km(
  p_order_id VARCHAR
) RETURNS NUMERIC AS $$
  SELECT ROUND(
    ST_Distance(
      sg.geom::geography, -- punto del vendedor
      cg.geom::geography  -- punto del cliente
    ) / 1000, 2           -- convertir metros → km
  ) AS distancia_km
  FROM   order_items oi
  JOIN   orders      o   ON o.order_id   = oi.order_id
  JOIN   sellers      s   ON s.seller_id  = oi.seller_id
  JOIN   customers    c   ON c.customer_id = o.customer_id
  JOIN   geolocation sg  ON sg.zip_code_prefix = s.zip_code_prefix
  JOIN   geolocation cg  ON cg.zip_code_prefix = c.zip_code_prefix
  WHERE  oi.order_id = p_order_id
  LIMIT  1;
$$ LANGUAGE sql STABLE;

-- Uso: obtener distancia de un pedido específico
SELECT fn_shipping_distance_km('00010242fe8c5a6d1ba2dd792cb16214');

-- Reporte: vendedores dentro de 50 km de São Paulo (capital)
SELECT s.seller_id, s.zip_code_prefix,
  ROUND(ST_Distance(sg.geom::geography,
    ST_MakePoint(-46.6333,-23.5505)::geography)/1000,2) AS
km_sao_paulo
FROM   sellers s
JOIN   geolocation sg ON sg.zip_code_prefix = s.zip_code_prefix
WHERE  ST_DWithin(sg.geom::geography,
  ST_MakePoint(-46.6333,-23.5505)::geography,
  50000) -- 50 km en metros
ORDER BY km_sao_paulo;

```

2.3 pg_trgm: Búsqueda de Productos Tolerante a Errores

El catálogo Olist contiene 32,951 productos con nombres en portugués. La extensión **pg_trgm** descompone cadenas en trigramas (grupos de 3 caracteres consecutivos) y calcula similitud entre 0 y 1. Un umbral de 0.3 captura errores tipográficos comunes sin sacrificar relevancia:

```
-- =====
-- pg_trgm: Búsqueda tolerante a errores tipográficos
-- =====

CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Índice GIN trigrama sobre nombre de categoría (portugués)
CREATE INDEX idx_products_category_trgm
  ON products USING GIN (product_category_name gin_trgm_ops);

-- Índice GIN trigrama sobre nombre de categoría en inglés
CREATE INDEX idx_category_name_en_trgm
  ON product_category USING GIN (product_category_name_english gin_trgm_ops);

-- Ajustar umbral de similitud (0.3 = permisivo, 0.6 = estricto)
SET pg_trgm.similarity_threshold = 0.3;

-- Q1: Buscar 'esporte' con error tipográfico ('ezporte')
SELECT product_category_name,
       similarity(product_category_name, 'ezporte_lazer') AS score
FROM   products
WHERE  product_category_name % 'ezporte_lazer' -- operador de similitud
ORDER BY score DESC
LIMIT 10;

-- Q2: Búsqueda full-text en categoría inglés ('electronics' → 'electronics')
SELECT product_category_name_english,
       similarity(product_category_name_english, 'electronics') AS score
FROM   product_category
WHERE  product_category_name_english % 'electronics'
ORDER BY score DESC;
```

2.4 pgcrypto: Protección de Datos Personales

En Ecommify, **customer_unique_id** identifica personas reales entre múltiples pedidos y **customer_zip_code_prefix** puede revelar ubicación. pgcrypto implementa hashing y cifrado en reposo:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Hash irreversible de customer_unique_id (pseudonimización)
ALTER TABLE customers
  ADD COLUMN customer_unique_id_hash VARCHAR(64);

UPDATE customers
  SET customer_unique_id_hash = encode(
    digest(customer_unique_id || 'ecommify_salt_2026', 'sha256'),
    'hex'
  );

-- Cifrado simétrico de zip_code (dato sensible de ubicación)
ALTER TABLE customers
  ADD COLUMN zip_encrypted BYTEA;

UPDATE customers
  SET zip_encrypted = pgp_sym_encrypt(
    customer_zip_code_prefix::TEXT,
    current_setting('app.encryption_key') -- key en GUC de la sesión
  );

-- Descifrar cuando se requiere (solo en capa de aplicación autenticada)
SELECT customer_id,
       pgp_sym_decrypt(zip_encrypted,
                       current_setting('app.encryption_key')) AS zip_code
FROM   customers
WHERE  customer_id = '06b8999e2fbala1fbc88172c00ba8bc7';
```

2.5 Matriz de Decisión Final — Extensiones_PostgreSQL_Ecommify

La siguiente tabla consolida la decisión de implementación para cada extensión evaluada, funcionando como el documento Extensiones_PostgreSQL_Ecommify.md requerido:

Extensión	Decisión	Justificación técnica y de negocio
PostGIS	Implementar	El dataset geolocation (lat/lng) es la clave para calcular distancia real vendedor→cliente. El flete actual es fijo; PostGIS permite tarifas dinámicas por kilómetro. Reemplaza JOIN costoso con geolocation por función ST_Distance. Índice GIST sobre geometría reduce búsqueda de vendedores cercanos de $O(n)$ a $O(\log n)$.
pg_trgm	Implementar	Los nombres de producto en portugués son propensos a errores tipográficos ('camiseta' vs 'camizeta'). similarity() devuelve valor 0-1; umbral 0.3 captura errores comunes. El índice GIN sobre trgm es 10-50x más rápido que ILIKE '%texto%'. Crítico para UX del buscador.
hstore	Implementar (parcial)	Ya integrado en Sección 1 para seller_metadata. Adecuado para metadatos operativos planos. No reemplaza JSONB para estructuras anidadas. Evaluación: mantener para sellers, no extender a products.
pgcrypto	Implementar	customer_unique_id identifica personas reales entre pedidos. pgcrypto::pgp_sym_encrypt() protege en reposo. crypt() para hashing de tokens.
pg_stat_statements	Implementar en dev/staging	Herramienta de diagnóstico, no de producción crítica. Permite identificar los top-10 queries por tiempo total antes de agregar índices. Overhead ~2-5% aceptable en entorno de análisis.

Tabla 5. Decisión final de extensiones PostgreSQL para Ecommify

3. Técnicas de Modelado para Cargas Híbridas OLTP/OLAP

La Actividad 1 propuso una arquitectura donde PostgreSQL actúa como sistema de registro transaccional (OLTP) y una réplica de solo lectura sirve las consultas analíticas (OLAP). Esta sección implementa esa propuesta con código DDL ejecutable, vistas materializadas, triggers de auditoría y un plan de mantenimiento operacional completo.

3.1 Identificación de Requisitos OLTP vs. OLAP en Ecommify

Antes de diseñar el esquema híbrido es necesario caracterizar los patrones de acceso de cada carga. El dataset Olist permite identificarlos con precisión a partir de sus volúmenes y relaciones:

Característica	OLTP (Transaccional)	OLAP (Analítico)
Volumen de escritura	Alto — pedidos, pagos, reseñas en tiempo real (hasta 100K+ TPS en escala)	Muy bajo — datos históricos; escritura solo en refresh de MVs
Patrón de lectura	Punto: 1 registro por PK (order_id, customer_id)	Agregación: millones de filas → GROUP BY, SUM, COUNT
Latencia requerida	< 10 ms — checkout, confirmación de pago	< 5 s — dashboards ejecutivos, reportes mensuales
Consistencia	ACID estricto — no se acepta doble cobro ni pedido sin cliente	Eventual — datos de ayer son suficientes para reportes
Índices óptimos	B-tree sobre PKs y FKs frecuentes en WHERE	B-tree en columnas de fecha + índices en columnas de agregación
Tablas calientes	orders (INSERT), order_items (INSERT), order_payments (INSERT)	orders_history (SELECT), mv_sales_by_category (SELECT)
Estrategia de acceso	Búsqueda por PK, FK lookup, JOINS pequeños (2-3 tablas)	Full scan con filtro de partición, hash joins de millones de filas

Tabla 6. Requisitos diferenciados OLTP vs. OLAP — Ecommify / Olist

3.2 Particionamiento de la Tabla orders por Fecha

La tabla **orders** (99,441 filas en Olist; millones en producción) es el núcleo de ambas cargas. El particionamiento por rango de **order_purchase_timestamp** permite que las consultas analíticas históricas escaneen solo la partición relevante (partition pruning) mientras las escrituras OLTP aterrizan en la partición activa:

```

-- Tabla principal particionada (reemplaza la versión de Actividad 1)
CREATE TABLE orders (
  order_id          VARCHAR(50)  NOT NULL,
  customer_id       VARCHAR(50)  NOT NULL,
  order_status      VARCHAR(20)  NOT NULL,
  order_purchase_timestamp  TIMESTAMP  NOT NULL,
  order_approved_at  TIMESTAMP,
  order_delivered_carrier_date  TIMESTAMP,
  order_delivered_customer_date  TIMESTAMP,
  order_estimated_delivery_date  TIMESTAMP,
  updated_at        TIMESTAMP    NOT NULL DEFAULT NOW(),
  CONSTRAINT pk_orders PRIMARY KEY (order_id, order_purchase_timestamp),
  CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id),
  CONSTRAINT chk_delivery_after_purchase CHECK (
    order_delivered_customer_date >= order_purchase_timestamp
    OR order_delivered_customer_date IS NULL)
) PARTITION BY RANGE (order_purchase_timestamp);

-- Particiones por año (dataset Olist: 2016-2018)
CREATE TABLE orders_2016 PARTITION OF orders
  FOR VALUES FROM ('2016-01-01') TO ('2017-01-01');

CREATE TABLE orders_2017 PARTITION OF orders
  FOR VALUES FROM ('2017-01-01') TO ('2018-01-01');

CREATE TABLE orders_2018 PARTITION OF orders
  FOR VALUES FROM ('2018-01-01') TO ('2019-01-01');

-- Partición DEFAULT para datos fuera de rango (siempre recomendada)
CREATE TABLE orders_default PARTITION OF orders DEFAULT;

-- Índices sobre la partición activa (OLTP write path)
CREATE INDEX idx_orders_2018_customer ON orders_2018(customer_id);
CREATE INDEX idx_orders_2018_status   ON orders_2018(order_status);

```

3.3 Vistas Materializadas para Dashboards

Las vistas materializadas pre-computan agregaciones costosas y las almacenan como tablas físicas. En Ecommify reemplazan queries de 5-6 JOINS con GROUP BY sobre la tabla completa de orders:

Vista Materializada	Contenido	Justificación de negocio	Índices internos
mv_sales_by_category_monthly	Ventas totales, número de pedidos y ticket promedio agrupados por categoría de producto y mes del año	Dashboard mensual de ingresos por categoría. Reemplaza un JOIN de 5 tablas con GROUP BY sobre 100K+ filas. Refresh semanal es suficiente para reportes ejecutivos.	UNIQUE (category_name_english, year, month). B-tree sobre year DESC, month DESC para filtros recientes.
mv_customer_segments	Segmentación RFM (Recencia, Frecuencia, Monto) de clientes únicos: días desde última compra, número de pedidos, gasto total	Módulo de marketing para campañas dirigidas. Evita recalcular RFM en cada query de CRM. Actualización semanal mantiene segmentos relevantes sin overhead en OLTP.	UNIQUE (customer_unique_id). B-tree sobre total_spent DESC para top-clientes. B-tree sobre recency_days para clientes inactivos.
mv_seller_performance	Métricas por vendedor: total de ítems despachados, valor promedio, tasa de entrega a tiempo, calificación promedio de reseñas	Panel de gestión de sellers marketplace. Identifica vendedores con bajo rendimiento o alto volumen. Refresh mensual alineado con ciclos de evaluación.	UNIQUE (seller_id). B-tree sobre avg_review_score, on_time_rate para rankings.
mv_delivery_analysis	Análisis de tiempos de entrega: promedio días prometidos vs. reales, tasa de entrega tardía	Reportes operativos de logística. Identifica estados con mayor retraso sin consultar 99K pedidos en tiempo real. Refresh semanal.	UNIQUE (seller_state, customer_state). B-tree sobre late_delivery_rate DESC.

Tabla 7. Vistas materializadas diseñadas para dashboards de Ecommify

```

-- =====
-- MV 1: Ventas por categoría y mes
-- =====
CREATE MATERIALIZED VIEW mv_sales_by_category_monthly AS
SELECT
    pc.product_category_name_english      AS category_name_english,
    EXTRACT(YEAR FROM o.order_purchase_timestamp) AS year,
    EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month,
    COUNT(DISTINCT o.order_id)              AS total_orders,
    COUNT(oi.order_item_id)                 AS total_items,
    ROUND(SUM(oi.price), 2)                  AS total_revenue,
    ROUND(AVG(oi.price), 2)                  AS avg_ticket,
    ROUND(SUM(oi.freight_value), 2)          AS total_freight
FROM    orders        o
JOIN    order_items    oi ON oi.order_id = o.order_id
JOIN    products       p  ON p.product_id = oi.product_id
JOIN    product_category pc ON pc.product_category_name =
p.product_category_name
WHERE   o.order_status = 'delivered'
GROUP BY pc.product_category_name_english,
         EXTRACT(YEAR FROM o.order_purchase_timestamp),
         EXTRACT(MONTH FROM o.order_purchase_timestamp)
WITH DATA;

CREATE UNIQUE INDEX ON mv_sales_by_category_monthly
    (category_name_english, year, month);
CREATE INDEX ON mv_sales_by_category_monthly (year DESC, month DESC);

```

```

-- =====
-- MV 2: Segmentación RFM de clientes (Recencia-Frecuencia-Monto)
-- =====
CREATE MATERIALIZED VIEW mv_customer_segments AS
SELECT
    c.customer_unique_id,
    COUNT(DISTINCT o.order_id)              AS total_orders,
    ROUND(SUM(op.payment_value), 2)          AS total_spent,
    ROUND(AVG(op.payment_value), 2)          AS avg_order_value,
    MAX(o.order_purchase_timestamp)          AS last_purchase_date,
    CURRENT_DATE - MAX(o.order_purchase_timestamp)::DATE AS recency_days,
    CASE
        WHEN COUNT(DISTINCT o.order_id) >= 5
            AND SUM(op.payment_value) > 1000 THEN 'Champion'
        WHEN COUNT(DISTINCT o.order_id) >= 3 THEN 'Loyal'
        WHEN MAX(o.order_purchase_timestamp) > NOW() - INTERVAL '90 days'
            THEN 'Recent'
        ELSE 'At Risk'
    END AS rfm_segment
FROM    customers      c
JOIN    orders          o  ON o.customer_id = c.customer_id
JOIN    order_payments  op ON op.order_id   = o.order_id
GROUP BY c.customer_unique_id

```

```

WITH DATA;

CREATE UNIQUE INDEX ON mv_customer_segments (customer_unique_id);
CREATE INDEX ON mv_customer_segments (total_spent DESC);
CREATE INDEX ON mv_customer_segments (recency_days);

```

3.4 Triggers de Auditoría: Campo updated_at

Los triggers garantizan que el campo **updated_at** se actualice automáticamente en cada modificación, sin depender de la lógica de aplicación. Este campo es crítico para el pipeline CDC (Change Data Capture) que sincroniza PostgreSQL con MongoDB:

```

-- =====
-- Función de trigger reutilizable para updated_at
-- =====
CREATE OR REPLACE FUNCTION fn_set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at := NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger en orders (OLTP principal)
CREATE TRIGGER trg_orders_updated_at
    BEFORE UPDATE ON orders
    FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();

-- Trigger en products (catálogo sincronizado con MongoDB)
CREATE TRIGGER trg_products_updated_at
    BEFORE UPDATE ON products
    FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();

-- Trigger en customers (datos personales – auditoría LGPD)
CREATE TRIGGER trg_customers_updated_at
    BEFORE UPDATE ON customers
    FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();

```

3.5 Plan de Mantenimiento: Jobs Programados

La sostenibilidad del esquema híbrido depende de tareas de mantenimiento programadas. El plan cubre VACUUM, refresh de vistas materializadas, creación de particiones y verificación de consistencia entre sistemas:

Job / Tarea	Frecuencia	Justificación técnica
VACUUM ANALYZE (orders, order_items)	Diario — 02:00 UTC	Recupera espacio de filas muertas por UPDATE de order_status. ANALYZE actualiza estadísticas del planificador de queries. Sin VACUUM, el bloat degrada índices B-tree en tablas de alta escritura.
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_sales_by_category_monthly	Semanal — lunes 03:00 UTC	CONCURRENTLY permite lectura simultánea durante el refresh (sin lock exclusivo). La MV de ventas se actualiza semanalmente porque los dashboards ejecutivos no requieren datos del mismo día.
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_customer_segments	Semanal — lunes 03:30 UTC	El segmento RFM cambia gradualmente. Refresh semanal sincronizado con campañas de marketing. El desfase de 7 días es aceptable para comunicaciones de CRM.
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_seller_performance, mv_delivery_analysis	Mensual — 1er día del mes 04:00 UTC	Métricas de evaluación de vendedores y logística se usan en ciclos mensuales. Refresh menos frecuente reduce overhead sin impactar decisiones operativas.
CREATE PARTITION orders_YYYY_MM	Mensual — último día del mes 23:00 UTC	Crea la partición del mes siguiente ANTES de que lleguen datos. Evita el error 'no partition of relation found' en producción. Script parametrizado con fecha calculada dinámicamente.
pg_partman: mantenimiento automático de particiones	Semanal — domingo 01:00 UTC	Extensión pg_partman gestiona creación y retención de particiones. Archiva particiones de más de 3 años a tablespace lento (cold storage) automáticamente.
Verificación de consistencia PostgreSQL ↔ MongoDB	Diario — 05:00 UTC	Job Python que compara COUNT(product_id) entre PostgreSQL y colección MongoDB de catálogo. Alerta vía Slack si la diferencia supera el 0.1% (posible fallo del pipeline CDC/Kafka).
Backup físico (pg_basebackup)	Diario — 01:00 UTC	Backup completo a almacenamiento externo. Retención 30 días. Complementado con WAL archiving continuo para recuperación punto-a-tiempo (PITR) con RPO < 5 min.

Tabla 8. Jobs programados de mantenimiento — Ecommify

3.6 Métricas de Monitoreo OLTP y OLAP

El plan de monitoreo define umbrales operacionales para detectar degradación de rendimiento antes de que impacte usuarios finales. Las métricas cubren ambas cargas y el pipeline de sincronización:

Capa	Métrica	Umbral objetivo	Fuente de medición
OLTP	Latencia de INSERT en orders	< 5 ms p99	pg_stat_statements — mean_exec_time para INSERT en orders
OLTP	Tasa de transacciones por segundo (TPS)	> 500 TPS sostenido	pg_stat_activity + contador xact_commit/xact_rollback en pg_stat_database
OLTP	Cache hit ratio de buffer	> 99%	pg_stat_bgwriter: blks_hit / (blks_hit + blks_read) × 100
OLTP	Tiempo de espera por locks	< 100 ms p95	pg_stat_activity WHERE wait_event_type = 'Lock'
OLAP	Tiempo de refresh de MVs	< 60 s por MV	Logear duration de REFRESH MATERIALIZED VIEW en pg_log
OLAP	Filas escaneadas en queries analíticos	< 10M filas por query	EXPLAIN ANALYZE — rows estimados vs. reales en nodos Seq Scan
OLAP	Tamaño de particiones activas	< 10 GB por partición mensual	pg_relation_size(partition_name) en jobs de monitoreo
Híbrido	Lag de CDC PostgreSQL → MongoDB	< 2 s	Debezium lag metrics en Kafka Consumer Group
Híbrido	Desviación de conteos PG vs. Mongo	< 0.1%	Job diario Python: abs(pg_count - mongo_count) / pg_count

Tabla 9. Métricas de monitoreo OLTP, OLAP e híbrido — Ecommify

3.7 Estrategia de Escalamiento — Free Tier a Producción

El dataset Olist representa un sistema de tamaño medio. La siguiente estrategia define acciones concretas cuando Ecommify alcanza los límites de cada nivel:

```
-- =====
-- Script de creación automática de partición mensual
-- Ejecutar como job al último día de cada mes
-- =====
DO $$
DECLARE
    v_next_month DATE := DATE_TRUNC('month', NOW()) + INTERVAL '1 month';
    v_month_end DATE := v_next_month + INTERVAL '1 month';
    v_part_name TEXT := 'orders_' || TO_CHAR(v_next_month, 'YYYY_MM');
BEGIN
    EXECUTE format(
```

```
'CREATE TABLE IF NOT EXISTS %I PARTITION OF orders
  FOR VALUES FROM (%L) TO (%L)',
  v_part_name, v_next_month, v_month_end
);
RAISE NOTICE 'Partición % creada: % → %',
  v_part_name, v_next_month, v_month_end;
END;
$$;
```

La estrategia de escalamiento progresivo contempla tres niveles alineados con el crecimiento esperado de Ecommify:

1. **Nivel 1 — Free Tier (< 10 GB, dataset Olist):** PostgreSQL en instancia única con particionamiento por año. MVs con refresh manual. Sin réplica. Monitoreo vía `pg_stat_statements`.
2. **Nivel 2 — Startup (10–100 GB):** Réplica de solo lectura para queries analíticos (`pg_basebackup` + WAL streaming). MVs con refresh automático vía `pg_cron`. Particionamiento mensual automático con el script anterior.
3. **Nivel 3 — Escala (> 100 GB):** Sharding de MongoDB para catálogo (shard key: `product_category`). PostgreSQL con Connection Pooler (PgBouncer). Migración de OLAP a capa dedicada (Redshift/BigQuery) con CDC continuo desde PostgreSQL.

Referencias

- PostgreSQL Global Development Group. (2025). Data types. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/datatype.html>
- PostgreSQL Global Development Group. (2025). Server programming: Extending SQL. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/extend.html>
- PostgreSQL Global Development Group. (2025). PostGIS manual. <https://postgis.net/documentation/>
- PostgreSQL Global Development Group. (2025). pg_trgm — support for similarity of text using trigram matching. <https://www.postgresql.org/docs/16/pgtrgm.html>
- PostgreSQL Global Development Group. (2025). pgcrypto — cryptographic functions. <https://www.postgresql.org/docs/16/pgcrypto.html>
- Brewer, E. A. (2000). Towards robust distributed systems. Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC), 7. <https://doi.org/10.1145/343477.343502>
- Kleppmann, M. (2017). Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. O'Reilly Media.
- Olist. (2018). Brazilian e-commerce public dataset by Olist [Dataset]. Kaggle. <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
- MongoDB, Inc. (2024). MongoDB manual. <https://www.mongodb.com/docs/manual/>